

Commercial Auto Risk Scoring

Case Study

Summary & Objective

Commercial auto insurance covers multi-vehicle enterprise fleets. Commercial fleets represent high-exposure risk pools.

Primary Objective:

Estimate the continuous probability of claim occurrence ($\text{claim} > 0$) for each `policy_id` using binary classification.

Approach:

Create an end-to-end ML pipeline predicting policy claim incidence ($\text{claim} > 0$) to prioritize triage and optimize loss ratios.

Problem Formulation & Key Assumptions

- Claims are rare events (12.3% in this scenario)
- The classes (claim / no claim) are very imbalanced
- Accuracy may be misleading in this scenario
 - returning high accuracy in identifying the 'no claim' policies and we may miss the 'claim' scenarios
- Binary classification vs. continuous probability
 - as loss is rare continuous probability may add more value / be more descriptive

The aim is to train the model rather to better identify these 'outlier' situations – to minimize the loss.

Exploratory Data Analysis & Data Integrity

- **Real data vs. Train dataset**
 - column inconsistencies: 'has_safety_program'
- **Data leakage prevention**
 - excluded from features
 - columns determined by target column 'claim_count'
 - missing column in real dataset: 'has_safety_program'
- **Deduplication**
 - exact row-level
 - primary key 'policy_id'
- **Data types**
 - mixed types / data in single columns: n/a
 - column's data type update: datetime
 - case formatting: business_type

	policy_id	business_type
52	POL-000567	trucking
12881	POL-000567	Trucking

Exploratory Data Analysis & Data Integrity

- **Outliers and anomalies**

- replaced:

- 'business_type': 'agriculture' with 'other' (1 row)

- dropped:

- chronological anomalies: expiration date < effective date (10 rows)
 - negative annual premiums (maybe cancellations): 'annual_premium' < 0 (5 rows)
 - ghost fleets: 'total_vehicles' = 0 with active drivers or mileage > 0 (2 rows)

- **Null (missing) values**

- 'prior_loss_amount'

- strategy: if 'prior_apd_claim_count' and 'prior_al_claim_count' = 0 then 0.0 else median

- 'payment_frequency'

- quite balanced cross categories; strategy: 'unknown'

- **Snapshot date**

- all records: during time when the policy is still open
 - 77 rows: 'snapshot_date' == 'policy_effective_date' --> elapsed time = 0
 - 11 rows: where 'claim_count' > 0

Feature engineering

- **exposure_ratio** = elapsed_days / policy_length_days
 - elapsed_days = snapshot_date - policy_effective_date
 - policy_length_days = policy_expiration_date - policy_effective_date
- **mileage_per_vehicle** =
'prior_year_mileage_000'/'total_vehicle_count'
 - total_vehicle_count = 'vehicle_count' + 'num_heavy_vehicles'
- **loss_per_claim**
= if total_prior_claims > 0 then prior_loss_amount / total_prior_claims else 0.0
- **has_large_prior_loss** = prior_loss_amount > 50000
- **risk_score_external_isna** = True/False - risk_score_external

Preprocessing & improvements

- Evaluate: use "X_test" / "y_test" instead of train data
- Cleanup of training / real data
- Feature engineering – adding new columns + remove target ones
- train_test_split data
- Create Pipeline
 - after train_test_split
 - define steps for imputation / encoding / scaling
 - 0 for all numeric values may not describe the reality
- Try various models
 - Selected: XGBoost
- Metrics
 - Accuracy may not be suitable in this case
 - Imbalanced classes

Modeling Strategy & Validation

- Model: XGBoost

- Captures non-linear operational interactions without manual feature combinations
- should be more resilient to skewness and outliers
- Native handling of sparse features and missing values.

```
# imbalanced claims ratio for XGBoost
scale_pos_weight = (len(y_train) - y_train.sum()) / y_train.sum()

model = XGBClassifier(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=4,
    subsample=0.8,
    colsample_bytree=0.8,
    # scale_pos_weight=scale_pos_weight, # tried by 1st run
    scale_pos_weight=1.0,
    eval_metric="logloss",
    random_state=42,
)
```

- Metrics Tracked:

- ROC-AUC & Gini: Evaluates risk rank-ordering.
- Brier Score: Assesses probability calibration accuracy.

Metric	1st	2nd
ROC-AUC:	0.6386	0.6436
Gini Score:	0.2773	0.2872
Brier Score:	0.1773	0.1066

Production Readiness Improvements

- file preparation / refactoring
 - remove unnecessary imports / prints
 - move functions into separate file(s)
 - put the code into .py file instead of ipynb
 - separate credentials into .env file - avoid hardcoding
 - include requirements.txt file (for compatibility)
- separate the flow:
 - file(s) for EAD
 - file(s) for preprocessing data
 - file(s) for training the model
 - file for model prediction
 - save model and all its parameters to file
- implement source schema validation
- add tests and logging

Production Readiness Roadmap

- code review
- prepare documentation
- commit to github
- orchestration of the jobs
- files (if any) save to cloud / official storage location
 - check rights
- after deployment
 - monitor the performance
 - re-train if needed or conditions have been changed